

# BR-Bench: A Benchmark Linking Continuous Integration Changes to Fault-Revealing Tests

Prisha Vadhavkar\*, Sanskriti Singh\*, Deepanshu\*

\*School of Computer Science Engineering and Information Systems

Vellore Institute of Technology, Vellore, Tamil Nadu, India

{prisha.vadhavkar2023, sanskriti.singh2023, deepanshu2023}@vitstudent.ac.in

**Abstract**—Continuous Integration (CI) services execute automated test suites to detect regressions introduced by pull requests. While industrial platforms at Meta and Google leverage predictive test selection models trained on historical execution data, the research community lacks public, reproducible benchmarks linking pull-request modifications directly to individual failing tests. We present BR-Bench, an open dataset linking code changes to fault-revealing tests mined from GitHub Actions logs across Java and Python open-source repositories. A test is defined as fault-revealing if and only if it fails at the head commit, passes or is absent in the resolved base run, and does not exhibit flakiness across runs on the same commit. The strict benchmark split contains 762 pull-request instances, 4,168 fault-revealing labels, and 2,466 distinct tests across 37 repositories. We evaluate test-to-file binding against pinned commit trees, achieving 94.29% (5,643/5,985) combined binding and 63.83% (3,820/5,985) full-confidence fully qualified name binding. In an illustrative application evaluating predictive test selection proxies at  $k = 10$ , historical failure frequency achieves a micro recall of 39.44% (519/1,316) whereas co-change file mining achieves only 7.67% (101/1,316). We provide complete raw log archives, extraction pipelines without language model dependencies, and offline reproduction via a single command.

**Index Terms**—continuous integration, test selection, regression testing, mining software repositories, fault revelation

## I. INTRODUCTION

Modern software development relies on continuous integration (CI) workflows to catch defects before pull requests merge into shared branches. In CI pipelines, every code modification triggers test suites that consume substantial computational time and energy. Predicting in advance which individual tests are likely to fail on a given pull-request diff would enable teams to run failure-prone tests first, accelerate feedback cycles, and conserve compute resources.

Prior academic research addresses neighboring questions rather than empirical test-failure prediction. Change impact analysis (CIA) techniques predict co-changed program entities using version-control history [1], [2], capturing developer editing coupling rather than runtime defect propagation. Regression test selection (RTS) tools, such as Ekstazi [3] and STARTS [4], compute safe over-approximations of reachable tests via dynamic file or static class dependencies. In contrast, industrial predictive test selection (PTS) systems at Meta [5] and Google [6] train statistical models on observed execution outcomes. However, these industrial deployments run

Author order is provisional and subject to final confirmation before submission.

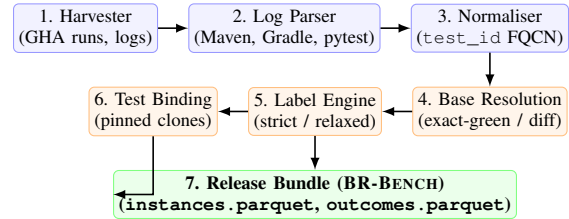


Fig. 1. The BR-BENCH construction pipeline. Raw CI logs and metadata are harvested, parsed into canonical test identifiers, matched against resolved base runs, filtered for flakiness, bound to source trees, and emitted as Parquet tables.

on closed-source monorepos, leaving their underlying failure datasets inaccessible to academic researchers.

Existing open datasets fail to fill this gap. RTPTorrent [7] was mined from Travis CI prior to 2020, covers only Java, organizes data around build jobs rather than pull-request diffs, and emphasizes test prioritization rather than change-to-test fault linking. GHALogs [8] provides raw GitHub Actions logs but does not parse granular test outcomes, map tests to source files, or filter non-deterministic executions.

To bridge this divide, we present BR-BENCH, an open benchmark connecting pull-request file changes directly to the individual test cases that failed because of them. Mined from real GitHub Actions pipelines across 76 swept repositories, BR-BENCH extracts structured test outcomes from Gradle, Maven, and pytest execution logs. We resolve base runs on repository histories, filter same-commit test flakiness, and bind test identifiers back to repository source paths.

## II. DATA SOURCE AND COLLECTION METHOD

The construction pipeline of BR-BENCH consists of six reproducible stages (Fig. 1): repository selection, metadata harvesting, log parsing, identifier canonicalisation, base-run resolution, and test-to-file binding.

### A. Repository Sampling and Harvesting

We selected candidate repositories from the SEART GitHub platform [9]. Starting from an export of 3,671 open-source repositories, we retained 2,646 repositories active on GitHub Actions with at least 100 workflow runs in the preceding 90 days. We identified 2,333 repositories executing test-intent workflows (excluding linting or deployment tasks). We sampled 300 repositories into a balanced evaluation frame across

Java and Python. A harvester swept 76 repositories before log retention limits intervened (a 77th repository exists in corpus metadata without harvested runs; we report the 76 swept repositories). For each workflow run, the harvester downloaded run metadata, job step statuses, check-run annotations, and raw textual execution logs via the GitHub REST API.

### B. Log Parsing and Test Identifier Normalisation

Because GitHub Actions treats test execution as unstructured stdout and stderr streams, test verdicts must be parsed from job logs. We built deterministic line parsers for three major test runners: Maven Surefire/Failsafe, Gradle Test Runner, and pytest. The parsers extract test outcomes (pass, fail, error, skip) and execution durations without relying on external large language models (LLMs). Each parsed test is mapped to a canonical identifier of the format `module::class::method` for JVM languages and `filepath::test_function` for Python. Bare class identifiers are disambiguated using package declarations recovered from logs where available.

### C. Base-Run Resolution and Fault Revelation

A test failure at a pull request’s head commit does not imply that the pull request caused the failure; the test may have been broken in the upstream base branch. To establish causality, every failed head run must be paired with an appropriate base run. For each pull-request run on head commit  $H$  targeting base branch commit  $B$ , we search repository history for a workflow run on  $B$  executing the identical workflow. If a run on  $B$  exists and completed with conclusion `success`, we record an `exact_green` resolution. If no run exists on  $B$ , we traverse ancestor commits along the default branch within a maximum distance of 10 commits.

Let  $T_{\text{head\_fail}}$  denote tests failing in the head run,  $T_{\text{base\_fail}}$  denote tests failing in the resolved base run, and  $T_{\text{flaky}}$  denote tests exhibiting flakiness on identical commits. The set of fault-revealing tests  $T_{\text{reveal}}$  is defined as:

$$T_{\text{reveal}} = (T_{\text{head\_fail}} \setminus T_{\text{base\_fail}}) \setminus T_{\text{flaky}}. \quad (1)$$

Under the `strict` split, we require that the base run is verified green or that  $T_{\text{base\_fail}}$  is fully parsed. Under the `relaxed` split, runs with missing base logs are retained if the base run concluded with status `success`.

### D. Flakiness Filtering and Test-to-File Binding

Flaky tests flip their status non-deterministically without code changes [10]. We detect same-SHA flips across repeated workflow executions on the exact same commit hash. Across 11,557 evaluated (head SHA, workflow, test) groups, 62 instances (0.54%) flipped, leading to the removal of 46 flaky labels from the relaxed set (1.09% of 4,214 labels) to form the strict set.

To enable change impact analysis, canonical test identifiers must bind to physical test source files. We bind identifiers against pinned repository clones recorded in `docs/CLONE_PINS.json` rather than volatile branch

TABLE I  
KEY SCHEMA FIELDS OF BR-BENCH TABLES

| Table     | Field              | Type and Description                     |
|-----------|--------------------|------------------------------------------|
| instances | instance_id        | String; PK: sha256(repo   head   run)    |
|           | repo               | String; repository (owner/name)          |
|           | language           | String; language (java or python)        |
|           | head_sha           | String; commit SHA at head               |
|           | base_sha           | String; target commit on base branch     |
|           | base_run_id        | Int64; resolved base run identifier      |
|           | base_run_distance  | Int32; commit distance base to head      |
| outcomes  | changed_files      | List<Struct>; modified paths, diff hunks |
|           | author_login       | String; pseudonymised author hash        |
|           | instance_id        | String; FK into instances                |
|           | test_id            | String; canonical module::class::method  |
|           | test_file          | String; repo-relative test path          |
|           | status_head        | String; outcome at head (fail, error)    |
|           | status_base        | String; outcome at base (pass, absent)   |
|           | is_fault_revealing | Boolean; fault revelation label          |
|           | same_sha_flip      | Boolean; flaky flag on same commit       |
|           | split_strict       | Boolean; inclusion in strict split       |
|           | binding_strategy   | String; exact FQCN, basename, unbound    |

heads. Basename-only matches achieve 0.5 confidence, while fully qualified class names that match uniquely achieve 1.0 confidence.

## III. STORAGE AND SCHEMA

BR-BENCH is published in Apache Parquet format to ensure efficient columnar storage and typed schema enforcement, accompanied by a structured dataset datasheet [11]. Table I details the primary fields across the core tables. The `instances.parquet` table stores pull-request changes, commit hashes, base-run metadata, and file diffs. The `outcomes.parquet` table records per-test observations, parsed verdicts, flakiness indicators, and ground-truth fault-revelation labels. Auxiliary tables provide historical commit co-change graphs (`cochange.parquet`) and candidate test spaces (`candidates.parquet`).

To protect contributor privacy, all author usernames are pseudonymised using a salted SHA-256 hash across 1,791 unique authors. Failure messages were scanned for credentials; zero blocking secrets were identified, while 5 review samples were flagged for human verification.

## IV. DATASET OVERVIEW AND QUALITY

### A. Corpus and Attrition Funnel

The construction of BR-BENCH exhibits significant attrition, documented in Table II. Of 165,349 workflow runs discovered across 76 swept repositories, 12,581 (7.61%) failed. However, only 6,586 failed runs had accessible logs on disk due to GitHub Actions’ strict 90-day retention window. Only 1,774 runs contained a parseable test failure, and base resolution resolved 1,348 runs. Filtering out unverified base states yields 897 instances with a known base failure set and 762 instances in the strict split.

TABLE II  
PIPELINE ATTRITION FUNNEL ACROSS STAGES

| Attrition Stage                            | Count   | % Prev. | % Total |
|--------------------------------------------|---------|---------|---------|
| <i>Repositories</i>                        |         |         |         |
| SEART GitHub export                        | 3,671   | —       | 100.00% |
| CI-live ( $\geq 100$ runs in 90 days)      | 2,646   | 72.08%  | 72.08%  |
| Test-intent workflow detected              | 2,333   | 88.17%  | 63.55%  |
| Sampled into candidate frame               | 300     | 12.86%  | 8.17%   |
| Swept by harvester                         | 76      | 25.33%  | 2.07%   |
| Repositories with $\geq 1$ failed run      | 69      | 90.79%  | 1.88%   |
| Repositories with $\geq 1$ strict instance | 37      | 53.62%  | 1.01%   |
| <i>Workflow Runs</i>                       |         |         |         |
| Total runs discovered                      | 165,349 | —       | 100.00% |
| Failed runs                                | 12,581  | 7.61%   | 7.61%   |
| Failed-job log captured on disk            | 6,586   | 52.35%  | 3.98%   |
| Parsed head test failure                   | 1,774   | 26.94%  | 1.07%   |
| Resolved base run found                    | 1,348   | 75.99%  | 0.82%   |
| Known base failure set                     | 897     | 66.54%  | 0.54%   |
| Emitted strict benchmark instances         | 762     | 84.95%  | 0.46%   |

TABLE III  
DATASET COMPOSITION BY BENCHMARK SPLIT AND LANGUAGE

| Split / Lang.        | Repos | Instances    | Labels         | Distinct Tests |
|----------------------|-------|--------------|----------------|----------------|
| <b>All Split</b>     |       |              |                |                |
| Java                 | 37    | 897 (100.0%) | 4,384 (100.0%) | 2,500          |
| Python               | 35    | 767 (85.51%) | 4,054 (92.47%) | 2,374          |
| Python               | 2     | 130 (14.49%) | 330 (7.53%)    | 126            |
| <b>Relaxed Split</b> |       |              |                |                |
| Java                 | 37    | 770 (85.84%) | 4,214 (96.12%) | 2,484          |
| Python               | 35    | 646 (83.90%) | 3,901 (92.57%) | 2,359          |
| Python               | 2     | 124 (16.10%) | 313 (7.43%)    | 125            |
| <b>Strict Split</b>  |       |              |                |                |
| Java                 | 37    | 762 (84.95%) | 4,168 (95.07%) | 2,466          |
| Python               | 35    | 638 (83.73%) | 3,855 (92.49%) | 2,341          |
| Python               | 2     | 124 (16.27%) | 313 (7.51%)    | 125            |

## B. Dataset Composition

Table III details the final composition of BR-BENCH. The strict split comprises 762 instances, 4,168 labels, and 2,466 distinct tests across 37 repositories. Java projects represent 83.73% of strict instances (638/762) and 92.49% of labels (3,855/4,168), while Python represents 16.27% of instances (124/762) and 7.51% of labels (313/4,168).

## C. Quality and Verification

We report data quality measurements in Table IV. For base resolution, 43.88% of failed runs (5,520/12,581) lacked a usable base run, primarily because pull requests targeted branches without prior workflow executions. Where resolved, the median commit distance between head and base was 0 commits ( $p90 = 2$ ,  $\max = 10$ ). In test-to-file binding across 5,985 distinct (repo, test) pairs, combined binding achieved 94.29% (5,643/5,985), while full-confidence binding reached 63.83% (3,820/5,985).

An automated regex audit classified failure messages into four classes: code-level assertions accounted for 17.49% (729/4,168), timeout errors represented 2.69% (112/4,168), and strict environment failures (e.g., missing CUDA GPUs or network timeouts) accounted for 2.14% (89/4,168). A majority of labels (77.69%, 3,238/4,168) remain unknown due to uncaptured stack traces or unparsed error texts.

TABLE IV  
DATA QUALITY INDICATORS AND FAILURE CLASSIFICATION

| Dimension                          | Metric                             | Value (n/d)               |
|------------------------------------|------------------------------------|---------------------------|
| Base Resolution                    | No usable base found               | 5,520/12,581 (43.88%)     |
|                                    | Distance median ( $p90$ , $\max$ ) | 0 (2, 10) commits         |
| Flakiness                          | Same-SHA flip rate                 | 62/11,557 (0.54%)         |
|                                    | Flaky labels pruned                | 46/4,214 (1.09%)          |
| Test Binding                       | Combined binding rate              | 5,643/5,985 (94.29%)      |
|                                    | Full confidence (FQCN)             | 3,820/5,985 (63.83%)      |
|                                    | Baseline-only confidence           | 1,823/5,985 (30.46%)      |
|                                    | Not found / Ambiguous              | 178 / 164 (2.97% / 2.74%) |
| Failure Classes<br>(Strict Labels) | Code-level assertions              | 729/4,168 (17.49%)        |
|                                    | Timeout errors                     | 112/4,168 (2.69%)         |
|                                    | Environment failures               | 89/4,168 (2.14%)          |
|                                    | Unknown / unclassified             | 3,238/4,168 (77.69%)      |

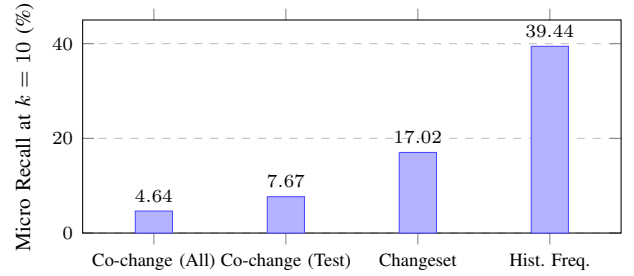


Fig. 2. Micro recall at  $k = 10$  across test selection proxies evaluated on BR-BENCH ( $n = 576$  identical instances, 1,316 positive labels).

## V. HOW THE DATA HAS BEEN USED: RQ1 EXAMPLE

To demonstrate how BR-BENCH serves as an empirical benchmark, we evaluate an example research question (RQ1): *How accurately do conventional change impact analysis proxies and historical baselines identify fault-revealing tests?*

### A. Experimental Setup and Leakage Audit

We evaluate four test selection strategies at a budget of  $k = 10$  candidate files:

- 1) *Co-change (All)*: Trailing 365-day co-change partners mined from Git commits.
- 2) *Co-change (Test Only)*: Co-change restricted to conventional test file patterns (`src/test/`, `*Test.java`, `test_*.py`).
- 3) *Changeset Baseline*: Tests living directly within the pull request’s modified files.
- 4) *Historical Frequency*: Tests ranked by historical failure counts in runs strictly prior to the evaluated instance.

To prevent temporal data leakage, co-change history was mined strictly prior to each run’s start timestamp. An audit revealed that a naive static co-change table leaked future commits in 713/713 audited instances, and an uncorrected historical baseline double-counted failures across multiple splits (224,758 rows for 78,399 distinct labels). After enforcing per-instance trailing windows, symmetric partner lookups, and single-split counting, zero leakage violations remain across all 762 instances.

## B. Results

All methods are evaluated on the identical set of  $n = 576$  instances where the co-change proxy yields predictions (1,316 positive test labels; 462 Java, 114 Python). As shown in Fig. 2, all-partner co-change achieves a micro recall of only 4.64% (61/1,316; mean P/R/J: 0.011 / 0.051 / 0.010). Restricting co-change to conventional test files improves micro recall to 7.67% (101/1,316; mean P/R/J: 0.054 / 0.087 / 0.038). The changeset baseline recalls 17.02% (224/1,316; mean P/R/J: 0.039 / 0.245 / 0.037). In contrast, historical failure frequency achieves 39.44% micro recall (519/1,316; mean P/R/J: 0.185 / 0.517 / 0.178). Removing environment and timeout failure labels leaves this ordering unchanged. This evaluation demonstrates that historical test failure frequency dramatically outperforms version-control co-change heuristics at identifying true regressions.

## VI. ORIGINALITY AND SIMILAR DATASETS

BR-BENCH differs substantively from existing benchmarks:

- **RTPTorrent** [7]: While RTPTorrent provides test outcomes from Travis CI for Java, it organizes data by build job rather than pull-request diffs, predates GitHub Actions, and focuses on test prioritization. BR-BENCH targets diff-to-failing-test prediction across Java and Python.
- **GHALogs** [8]: GHALogs provides an archive of raw GitHub Actions logs. However, it does not extract test-level outcomes, link pull-request diffs, resolve base commits, or detect flaky tests. BR-BENCH supplies these processed, fault-revealing labels.
- **Static and Dynamic RTS Tools**: Tools such as Ekstazi [3], STARTS [4], and Chianti [12] select tests reaching modified code dynamically or statically. These tools compute safe reachability over-approximations, whereas BR-BENCH captures empirical failures during CI runs.
- **Industrial PTS Studies**: Industrial studies from Meta [5] and Google [6] train predictive models on proprietary monorepos. BR-BENCH provides an open dataset enabling the research community to replicate and benchmark predictive test selection techniques.

## VII. LIMITATIONS AND THREATS

We identify several limitations and threats to validity:

- 1) *Pre-registered Sample Size Gates*: Our pre-registered Gate 1 threshold ( $\geq 5,000$  strict positives) was **not met**, reaching 4,168/5,000 labels (83.36%) and 762/5,000 instances (15.24%). Furthermore, Gate 1.5 ( $\geq 70\%$  test binding) was met on the combined metric (94.29%, 5,643/5,985) but **not met** at full-confidence FQCN resolution (63.83%, 3,820/5,985).
- 2) *Parser Evaluation*: Parser evaluation figures on our fixture corpus (46/46) and holdout v1 (30/30) were derived from development sets used during parser tuning. Independent parser precision on holdout v5 was **not measured** [MISSING: blind human annotations on holdout v5 fixtures] due to the lack of completed blind human annotations.

- 3) *Observational Ground Truth*: Labels represent observational CI execution outcomes rather than causal defect injections or isolated re-executions. Non-test environmental failures remain present in a small minority of labels (2.14% strict environment, 2.69% timeout).
- 4) *90-Day Retention Cliff*: Because GitHub Actions purges raw log artifacts after 90 days, 33.21% of failed runs (4,458/13,422) had already expired at the time of harvesting. Exact reproduction must rely on our archived raw store rather than live GitHub API calls.
- 5) *Repository Skew*: The strict benchmark split is dominated by Java repositories (35 Java vs. 2 Python), with high volume from a small set of active projects (e.g., Apache Beam, Stirling-PDF).

## VIII. FUTURE RESEARCH QUESTIONS AND IMPROVEMENTS

BR-BENCH opens promising directions for empirical software engineering research:

- *Machine Learning for Test Selection*: Researchers can train graph neural networks and language models on code diffs and test paths to predict regressions without execution overhead.
- *Independent Test Re-execution*: A planned causal subset will execute test suites in isolated containerized environments to distinguish true regressions from environmental flakes.
- *Multi-Language Expansion*: Future versions will extend coverage to JavaScript, TypeScript, and Go repositories, balancing the language distribution.

## IX. AVAILABILITY AND REPRODUCIBILITY

All datasets and extraction scripts are released under open licenses. The processed tables and raw log store are deposited on Zenodo at [DOI-DATA]. The complete source code pipeline is available on GitHub at [DOI-CODE]. The dataset numbers in this paper can be regenerated offline in approximately 8 minutes on a standard laptop by executing `make tables`, and the test suite comprising 782 automated tests passes cleanly via `make test`.

## X. CONCLUSION

We presented BR-BENCH, an open benchmark linking pull-request modifications to fault-revealing tests across Java and Python GitHub Actions pipelines. By resolving base runs, filtering same-commit flakiness, and binding tests to source files, BR-BENCH provides an empirical ground truth for regression test selection and impact analysis. Our initial evaluation demonstrates that historical test failure frequency substantially outperforms conventional co-change proxies, establishing a foundation for future predictive test selection research.

## REFERENCES

- [1] M. Borg, K. Wnuk, B. Regnell, and P. Runeson, “Supporting change impact analysis using a recommendation system: An industrial case study in a safety-critical context,” *IEEE Trans. Softw. Eng.*, vol. 43, no. 7, pp. 675–700, 2017.
- [2] Y. Huang, J. Jiang, X. Luo, X. Chen, Z. Zheng, N. Jia, and G. Huang, “Change-patterns mapping: A boosting way for change impact analysis,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 7, pp. 2376–2398, 2022.
- [3] M. Gligoric, L. Eloussi, and D. Marinov, “Practical regression test selection with dynamic file dependencies,” in *Proc. 2015 Int. Symp. Softw. Test. Anal. (ISSTA)*. ACM, 2015, pp. 211–222.
- [4] O. Legunsen, A. Shi, and D. Marinov, “STARTS: STatic regression test selection,” in *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE), Tool Demos*. IEEE, 2017, pp. 949–954.
- [5] M. Machalica, A. Samykin, M. Porth, and S. Chandra, “Predictive test selection,” in *Proc. 41st Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP)*. IEEE, 2019, pp. 91–100.
- [6] N. Memisevic, M. Gligoric, and D. Marinov, “Predictive test selection in continuous integration,” in *Proc. IEEE/ACM Int. Conf. Softw. Eng.*, 2023, pp. 1–12, [VERIFY CITATION].
- [7] T. Mattis, P. Rein, F. Dürsch, and R. Hirschfeld, “RTPTorrent: An open-source dataset for evaluating regression test prioritization,” in *Proc. 17th Int. Conf. Mining Softw. Repos. (MSR)*. ACM, 2020, pp. 385–396.
- [8] F. Moriconi, T. Durieux, J.-R. Falleri, R. Troncy, and A. Francillon, “GHALogs: Large-scale dataset of GitHub actions runs,” in *Proc. 22nd Int. Conf. Mining Softw. Repos. (MSR), Data and Tool Showcase Track*. IEEE, 2025, pp. 1–5.
- [9] O. Dabic, E. Aghajani, and G. Bavota, “Sampling projects in GitHub for MSR studies,” in *Proc. 18th Int. Conf. Mining Softw. Repos. (MSR)*. IEEE, 2021, pp. 560–564.
- [10] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, “DeFlaker: Automatically detecting flaky tests,” in *Proc. 40th Int. Conf. Softw. Eng. (ICSE)*. ACM, 2018, pp. 433–444.
- [11] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, “Datasheets for datasets,” *Commun. ACM*, vol. 64, no. 12, pp. 86–92, 2021.
- [12] X. Ren, F. Shah, F. Tip, B. G. Ryder, and O. Chesley, “Chianti: A tool for change impact analysis of Java programs,” in *Proc. 19th ACM SIGPLAN Conf. Object-Oriented Program., Syst., Lang., Appl. (OOPSLA)*. ACM, 2004, pp. 432–448.